

Course Outline: Web Accessibility Fundamentals

Title: Web Accessibility Fundamentals **Prerequisite:** HTML and CSS Basics (Week 0 prework) **Target Audience:** Beginner developers learning to create inclusive web experiences **Total Lessons:** 19 **Estimated Total Length:** ~9,500 words **Format:** Mixed (37% hands-on)

Course Description

Web accessibility ensures that websites and applications are usable by everyone, including people with disabilities. This course introduces the fundamental principles and practical techniques for creating accessible web experiences. Students will learn how assistive technologies interact with web content, master ARIA attributes and roles, and identify common accessibility problems.

Building on HTML and CSS knowledge from prework, this course focuses on the specific practices that make websites accessible. Students will learn to audit existing sites, implement accessibility features, and test their work using real tools. By the end of this course, students will understand both the "why" and "how" of web accessibility.

This course emphasizes practical application aligned with 4Geeks principles: students will learn the path of least resistance (semantic HTML first, ARIA when needed), develop attention to detail (accessibility requires precision), and build autonomous problem-solving skills (testing and auditing their own work).

Course Philosophy

- This course emphasizes:
- **Semantic HTML first:** The foundation of accessibility is using the right HTML elements for their intended purpose
 - **User-centered thinking:** Understanding how real people with disabilities interact with web content
 - **Practical testing:** Students will use actual accessibility tools and screen readers to verify their work
 - **Progressive enhancement:** Start with solid HTML structure, then enhance with ARIA when truly needed

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome to Web Accessibility	1	~450
01 - Accessibility Foundations	4	~2,100
02 - Making HTML Accessible	4	~2,200
03 - ARIA Attributes and Roles	4	~2,200
04 - Common Accessibility Problems	4	~1,900
05 - Testing and Assessment	2	~1,400
06 - Building Accessible Websites	1	~450
Total	19	~9,500

Section 00: Welcome to Web Accessibility

00.0 Why Accessibility Matters 📖 (~450 words)

Learning Objectives:

- Understand what web accessibility means and why it's essential
- Recognize the diverse range of users who benefit from accessible design
- Connect accessibility practices to real-world impact

Content Outline:

- Definition of web accessibility
- Statistics about disability and web usage
- Types of disabilities that affect web interaction (visual, auditory, motor, cognitive)
- How accessibility benefits all users (keyboard navigation, captions, clear language)
- Introduction to WCAG (Web Content Accessibility Guidelines) as industry standard
- The business case: legal requirements, market reach, and SEO benefits

Transitions: This introductory lesson sets the foundation for understanding why accessibility matters. The next section will explore the core principles that guide all accessibility work.

Key Principles:

- Accessibility is not an edge case—it affects 15-20% of the global population
- Good accessibility practices improve usability for everyone
- Accessibility must be built in from the start, not added as an afterthought
- Legal compliance (ADA, Section 508) is the minimum, not the goal

Examples:

- A user with low vision using screen magnification to read content
 - A deaf user relying on video captions to understand tutorial content
 - A user with motor disabilities navigating a website using only keyboard
 - An elderly user benefiting from high-contrast text and larger click targets
-

Section 01: Accessibility Foundations

01.0 Core Accessibility Principles (POUR) 📖 (~500 words)

Learning Objectives:

- Master the four POUR principles that guide all accessibility work
- Apply POUR principles to evaluate website features
- Recognize how POUR principles connect to practical implementation

Content Outline:

- The POUR framework: Perceivable, Operable, Understandable, Robust
- **Perceivable:** Information must be presentable to users in ways they can perceive
 - Text alternatives for images
 - Captions for audio/video
 - Adaptable content that works in different presentations
 - Distinguishable content (color contrast, audio control)
- **Operable:** Interface components must be operable by all users
 - Keyboard accessibility
 - Sufficient time to read/interact
 - Seizure prevention (no flashing content)
 - Navigable structure
- **Understandable:** Information and interface operation must be understandable
 - Readable text

- Predictable behavior
- Input assistance (error prevention and correction)
- **Robust:** Content must work with current and future assistive technologies
 - Valid HTML
 - Proper use of semantic elements
 - Compatibility with assistive technologies

Transitions: Understanding these four principles provides the framework for all accessibility decisions. Next, we'll explore the assistive technologies that users rely on to interact with web content.

Key Principles:

- POUR is a mental model, not a checklist—each principle informs design decisions
- All four principles must work together for true accessibility
- These principles apply regardless of specific technologies or design trends
- POUR helps prioritize when making trade-offs in design

Examples:

- **Perceivable failure:** An image-only navigation menu (screen reader users can't perceive it)
 - **Operable failure:** A dropdown that only works on mouse hover (keyboard users can't operate it)
 - **Understandable failure:** Error messages that say "Invalid input" without explaining what's wrong
 - **Robust failure:** Custom interactive widgets built without proper ARIA attributes
-

01.1 Understanding Assistive Technologies 📖 (~550 words)

Learning Objectives:

- Identify the major categories of assistive technologies users employ
- Understand how screen readers interpret and announce web content
- Recognize the importance of keyboard-only navigation

Content Outline:

- **Screen readers:** Software that reads webpage content aloud
 - Popular screen readers: NVDA (Windows, free), JAWS (Windows, paid), VoiceOver (macOS/iOS, built-in), TalkBack (Android, built-in)
 - How screen readers navigate: headings, landmarks, links, forms
 - The "tab order" and focus management
 - Announcement patterns: link text, button labels, form fields
- **Screen magnification:** Software that enlarges portions of the screen
 - Users may only see a small portion of the page at once
 - Implications for layout and content organization
- **Keyboard-only navigation:** Users who cannot use a mouse
 - Tab key for moving between interactive elements
 - Enter/Space for activation
 - Arrow keys for some widgets
 - Importance of visible focus indicators
- **Voice control:** Dictation software and voice commands
 - Dragon NaturallySpeaking, Voice Control (macOS)
 - Clicking elements by speaking their visible labels
- **Switch devices:** Hardware switches for users with severe motor disabilities
 - Binary input (on/off) used to navigate and activate
 - Requires excellent keyboard accessibility as foundation

Transitions: Now that you understand the tools users rely on, we'll examine the legal and ethical landscape that makes accessibility not just good practice, but often a legal requirement.

Key Principles:

- Screen readers rely entirely on semantic HTML and proper ARIA attributes
- Keyboard navigation is the foundation—many assistive technologies build on it
- What appears visually on screen may differ significantly from what assistive technology conveys
- Assistive technologies are sophisticated tools used by expert users

Examples:

- A screen reader announcing: "Main navigation, region. Home, link. About, link. Contact, link."
 - A screen magnification user seeing only the top-left corner of a webpage, missing important navigation in the top-right
 - A keyboard-only user pressing Tab repeatedly to reach a "Skip to main content" link
 - A voice control user saying "Click submit button" to activate a form
-

01.2 Legal and Ethical Considerations 📖 (~500 words)

Learning Objectives:

- Understand major accessibility laws and standards
- Recognize the ethical imperative beyond legal compliance
- Identify the business benefits of accessible design

Content Outline:

- **Legal landscape:**
 - Americans with Disabilities Act (ADA) - applies to US businesses
 - Section 508 - applies to US federal agencies and contractors
 - European Accessibility Act (EAA) - EU requirements
 - WCAG as the technical standard referenced by laws worldwide
- **WCAG conformance levels:**
 - Level A: Minimum conformance (basic accessibility)
 - Level AA: Industry standard target (most legal requirements)
 - Level AAA: Enhanced accessibility (often not fully achievable)
- **The ethical case:**
 - The web as a fundamental human right (UN recognition)
 - Digital inclusion and equal access to information
 - Impact on education, employment, and civic participation
- **Business benefits beyond compliance:**
 - Larger market reach (15-20% of population has disabilities)
 - Improved SEO (semantic HTML, descriptive text)
 - Better usability for all users (mobile users, situational disabilities)
 - Reduced legal risk (lawsuits have increased dramatically)
 - Corporate social responsibility and brand reputation

Transitions: Understanding why accessibility matters sets the stage for practical application. In the next lesson, you'll learn to audit a website to identify accessibility issues.

Key Principles:

- Legal compliance is the floor, not the ceiling—aim for genuine usability
- Accessibility is a civil rights issue, not just a technical requirement
- Most accessibility improvements benefit all users, not just those with disabilities
- Retrofitting accessibility is expensive; building it in from the start is efficient

Examples:

- Domino's Pizza lawsuit (2019): Supreme Court ruled websites must be accessible under ADA
- Target settlement (2008): \$6 million for inaccessible website

- Netflix captioning requirements: FCC mandate for streaming video
 - The "curb cut effect": Curb cuts designed for wheelchairs benefit strollers, luggage, bicycles, and more
-

01.3 Auditing a Website for Accessibility (~550 words)

Learning Objectives:

- Conduct a basic accessibility audit using browser tools
- Identify common accessibility issues by inspection
- Interpret automated accessibility testing results
- Understand the limitations of automated testing

Content Outline:

- **Manual inspection checklist:**
 - Can you navigate the entire site using only the keyboard?
 - Are focus indicators clearly visible?
 - Does all non-text content have text alternatives?
 - Is the heading structure logical and hierarchical?
 - Do links have descriptive text (not "click here")?
 - Are form fields properly labeled?
 - Is color contrast sufficient?
- **Browser developer tools:**
 - Chrome DevTools Lighthouse accessibility audit
 - Firefox Accessibility Inspector
 - Edge Accessibility insights
- **Automated testing tools:**
 - axe DevTools (browser extension)
 - WAVE (Web Accessibility Evaluation Tool)
 - Understanding automated testing reports: errors, warnings, features to review
- **Limitations of automated testing:**
 - Automated tools catch only ~30-40% of accessibility issues
 - Cannot evaluate meaningful alt text (only detect if it exists)
 - Cannot assess logical heading order (only detect if headings exist)
 - Cannot determine if interactions make sense
- **Manual testing remains essential:**
 - Keyboard navigation testing
 - Screen reader testing (basic verification)
 - Color contrast checking in context
 - Logical flow and understandability

Transitions: You've now learned to identify accessibility problems. The next section will teach you how to fix them by making HTML itself more accessible.

Key Principles:

- Automated testing is a starting point, not a complete solution
- The best accessibility testing includes real users with disabilities
- Testing should happen throughout development, not just at the end
- Many accessibility issues are obvious once you start looking for them

Examples:

- Running Lighthouse and seeing a score of 68/100 with 12 accessibility issues flagged
- Using WAVE and discovering images without alt attributes highlighted in red
- Attempting to navigate a site with Tab key and getting stuck in a modal dialog
- Finding a form with unlabeled fields that screen readers announce as "Edit, blank"

Hands-On Exercise:

Practice Challenge: Audit an existing website for accessibility

Scenario: Choose a popular website you use regularly (news site, social media, e-commerce, etc.). Conduct a basic accessibility audit using the manual checklist and one automated tool.

Tasks:

1. Visit the website and attempt to navigate using only your keyboard (Tab, Shift+Tab, Enter, Escape)
2. Identify at least 3 keyboard navigation issues (if any)
3. Install the WAVE browser extension or use Chrome Lighthouse
4. Run an automated accessibility audit
5. Review the results and categorize issues by POUR principle
6. Write a brief report (200-300 words) summarizing:
 - The website you tested
 - The most significant accessibility barriers you found
 - Which POUR principle(s) each issue violates
 - One positive accessibility feature the site implements well

Success Criteria:

- Successfully completed keyboard-only navigation test
- Ran at least one automated audit tool
- Identified and correctly categorized at least 5 accessibility issues
- Connected issues to specific POUR principles
- Report demonstrates understanding of accessibility concepts

Expected Outcomes: Students will gain practical experience identifying real accessibility problems and begin developing the habit of thinking about accessibility when evaluating websites.

Section 02: Making HTML Accessible

02.0 How Screen Readers Interpret HTML 📖 (~500 words)

Learning Objectives:

- Understand how screen readers build a mental model from HTML structure
- Recognize the importance of semantic HTML elements
- Learn how screen readers navigate using landmarks, headings, and links

Content Outline:

- **The screen reader's view of a webpage:**
 - Not visual—a linear stream of content
 - Navigation by element type (headings, links, form fields)
 - Landmarks create a "table of contents"
 - Skip navigation mechanisms
- **Semantic HTML elements and their roles:**
 - `<header>`, `<nav>`, `<main>`, `<article>`, `<aside>`, `<footer>` create landmarks
 - `<h1>` through `<h6>` create navigable outline
 - `<a>` links are listed and navigable
 - `<button>` vs `<div>` with click handlers (only button is keyboard accessible)
 - `<form>`, `<label>`, `<input>` create proper form structure
- **How screen readers announce elements:**
 - Headings: "Heading level 2, Getting Started"
 - Links: "Link, Read more about accessibility"

- Buttons: "Button, Submit form"
- Images: "Image, [alt text content]"
- Form fields: "Email, edit, blank" (label, type, current value)
- **Navigation shortcuts screen reader users employ:**
 - H key: Jump to next heading
 - 1-6 keys: Jump to specific heading level
 - K/L: Navigate links
 - F: Navigate form fields
 - R: Navigate landmarks/regions

Transitions: Understanding how screen readers interpret HTML explains why semantic HTML is the foundation of accessibility. Next, you'll practice implementing accessible heading navigation.

Key Principles:

- Semantic HTML provides built-in accessibility that ARIA cannot fully replicate
- Screen reader users navigate by structure, not by scrolling visually
- The visual layout may differ completely from the semantic order
- Using the correct HTML element is always preferable to making a generic element behave correctly

Examples:

- A `<button>` element automatically: receives keyboard focus, activates on Enter/Space, announces as "button"
- A `<div onclick="...">` requires: `tabindex`, key handlers, `role="button"`, and still might not work correctly
- A well-structured page with `<main>`, `<nav>`, and proper headings allows a screen reader user to understand the page in seconds
- A page with only `<div>` elements requires the user to listen to every word to understand structure

02.1 Accessible Heading Navigation 📖 (~550 words)

Learning Objectives:

- Create a logical heading hierarchy for screen reader navigation
- Avoid common heading structure mistakes
- Use headings for structure, not for styling

Content Outline:

- **Heading hierarchy rules:**
 - One `<h1>` per page (usually the main page title)
 - Don't skip heading levels (`<h1>` → `<h3>` violates hierarchy)
 - Headings should nest logically (subsections use higher numbers)
 - Visual size doesn't determine heading level—structure does
- **Correct heading patterns:**

```
<h1>Introduction to Web Accessibility</h1>
<h2>What is Accessibility?</h2>
  <h3>Legal Requirements</h3>
  <h3>Ethical Considerations</h3>
<h2>Who Benefits?</h2>
  <h3>Visual Disabilities</h3>
  <h3>Motor Disabilities</h3>
```

- **Common mistakes:**
 - Using headings for styling ("I need big text, so I'll use `<h2>`")
 - Skipping levels for visual design
 - Multiple `<h1>` elements on a page

- No headings at all (relying on bold text or large divs)
- **Styling headings correctly:**
 - Use CSS to make any heading any size
 - Separate semantic meaning from visual presentation
 - Example: `h2 { font-size: 1rem; }` is perfectly valid if semantically appropriate

Transitions: Proper heading structure creates a navigable outline. Next, you'll learn to write effective alt text and link context.

Key Principles:

- Headings create an outline that screen reader users navigate like a table of contents
- The visual hierarchy and semantic hierarchy should match, but size is controlled by CSS
- Screen reader users often navigate by jumping from heading to heading
- Skipping heading levels confuses the logical structure of the page

Examples:

- **Good:** An article with `<h1>` for title, `<h2>` for sections, `<h3>` for subsections
- **Bad:** Using `<h3>` because it's the right size, even though it's the main page heading
- **Good:** A blog homepage with `<h1>` for site name, `<h2>` for each article title
- **Bad:** Multiple `<h1>` elements because each article feels important

Hands-On Exercise:

Practice Challenge: Fix the heading structure of a poorly organized page

Scenario: You've been given an HTML page with accessibility issues. The headings are chosen based on visual size rather than semantic meaning, resulting in a confusing structure for screen reader users.

Starting HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Coffee Shop Blog</title>
</head>
<body>
  <h3>Bean & Brew Coffee Blog</h3>

  <h1>Latest Posts</h1>

  <h2>The Best Coffee Brewing Methods</h2>
  <p>Discover our favorite ways to brew the perfect cup...</p>

  <h4>Pour Over Technique</h4>
  <p>Pour over coffee brings out subtle flavors...</p>

  <h4>French Press Method</h4>
  <p>For a bold, rich cup, try the French press...</p>

  <h1>About Our Shop</h1>
  <p>We've been serving specialty coffee since 2015...</p>

  <h5>Our Mission</h5>
  <p>To bring ethically sourced, perfectly roasted coffee...</p>
</body>
</html>
```

Tasks:

1. Identify all heading hierarchy violations
2. Rewrite the headings to create a logical, accessible structure
3. Ensure no heading levels are skipped
4. Maintain only one `<h1>` element per page
5. Test your structure: list out the heading hierarchy to verify it makes sense

Success Criteria:

- Single `<h1>` element (main page title)
- No skipped heading levels
- Logical nesting (subsections have higher heading numbers than their parent sections)
- The heading outline makes sense when read sequentially
- Visual design can be maintained through CSS (not required in this exercise)

Expected Outcomes: Students will understand that heading levels represent structure, not visual size, and will practice creating logical document outlines.

02.2 Alt Text Best Practices and Link Context (~575 words)

Learning Objectives:

- Write effective alt text that conveys meaning, not just description
- Identify when to use empty alt attributes
- Create descriptive link text that makes sense out of context

Content Outline:

- **Alt text fundamentals:**
 - Alt text conveys the purpose/meaning of an image, not necessarily its appearance
 - Ask: "What information does this image communicate?"
 - For decorative images: use empty alt (`alt=""`) so screen readers skip them
 - For functional images (buttons, links): describe the action/destination
 - Context matters: the surrounding text affects what alt text should say
- **Alt text patterns:**
 - **Informative images:** Describe the information conveyed
 - Bad: `alt="graph"`
 - Good: `alt="Bar graph showing 40% increase in sales from 2022 to 2023"`
 - **Decorative images:** Use empty alt
 - ``
 - **Functional images:** Describe the function
 - Bad: `alt="magnifying glass"`
 - Good: `alt="Search"`
 - **Complex images:** Brief alt + long description in text or via link
- **Link context and descriptive link text:**
 - Links should make sense when read out of context (screen readers can list all links)
 - Avoid: "Click here", "Read more", "Learn more"
 - Include destination or action in link text
 - Bad: Read more `<a href="...">here</a>`
 - Good: `<a href="...">Read more about accessibility testing</a>`
- **When generic links are unavoidable:**
 - Provide context through surrounding text
 - Use `aria-label` to give links unique names
 - Example: `<a href="..." aria-label="Read more about accessibility testing">Read more</a>`

Transitions: Images and links are two of the most common elements that need accessibility attention. Next, you'll learn to make forms accessible through proper labels and error messaging.

Key Principles:

- Alt text should convey meaning, not create redundancy
- Context determines appropriate alt text—the same image might need different alt text in different situations
- Every link should make sense if read alone, without surrounding context
- Screen readers can generate lists of all links on a page—generic "click here" links are useless in this list

Examples:

- **Bad alt text:** `alt="A dog"` for a service dog training photo in an article about accessibility
- **Good alt text:** `alt="Golden retriever wearing service dog vest assists person in wheelchair"`
- **Bad link:** "To learn about our services, click [here](#)"
- **Good link:** "[Learn about our accessibility consulting services](#)"

Hands-On Exercise:

Practice Challenge: Write effective alt text and fix link context

Scenario: You're auditing an article page with multiple images and links. The alt text is either missing or poorly written, and the links lack context.

Tasks:

Part 1: Fix alt text For each image below, write appropriate alt text or use empty alt if decorative:

1. Image: A screenshot showing a red error message "Required field" below an empty email input

- Current: `alt="screenshot"`
- Your alt text: _____

2. Image: Decorative geometric pattern used as a section divider

- Current: `alt="pattern"`
- Your alt text: _____

3. Image: Twitter icon linking to company Twitter page

- Current: `alt="twitter"`
- Your alt text: _____

4. Image: Pie chart showing browser market share (Chrome 65%, Firefox 20%, Safari 10%, Edge 5%)

- Current: `alt="pie chart"`
- Your alt text: _____

Part 2: Fix link context Rewrite these HTML snippets to provide proper link context:

1. Current:

```
<p>Our team has 10 years of experience. <a href="/about">Click here</a> for more information.</p>
```

Your fix: _____

2. Current:

```
<h2>Accessibility Audit Service</h2>
<p>We'll test your site for WCAG compliance.</p>
<a href="/services/audit">Learn more</a>
```

```
<h2>Training Workshops</h2>
<p>We offer hands-on accessibility training.</p>
<a href="/services/training">Learn more</a>
```

Your fix: _____

Success Criteria:

- Alt text conveys the information or function of each image
- Decorative images use empty alt (`alt=""`)
- Each link makes sense when read alone
- No "click here" or generic "learn more" links without context
- Functional images (icons) describe the function, not the image

Expected Outcomes: Students will practice writing meaningful alt text and creating descriptive links that are useful for screen reader users.

02.3 Accessible Forms: Labels and Error Messages (~575 words)

Learning Objectives:

- Associate form labels properly with input fields
- Provide clear, helpful error messages
- Implement accessible required field indicators

Content Outline:

- **Form label fundamentals:**
 - Every input must have an associated `<label>`
 - Use the `for` attribute to connect label to input
 - Screen readers announce: "[Label text], [input type], [current value]"

```
<label for="email">Email address</label>
<input type="email" id="email" name="email">
```

- **Placeholder vs. label** (common mistake):
 - Placeholders are not substitutes for labels
 - Placeholders disappear when typing begins
 - Not all screen readers announce placeholders
 - Labels persist and are always available
- **Required fields:**
 - Use the `required` attribute for browser validation
 - Indicate required fields visually AND in label text
 - Bad: Only using color or asterisk
 - Good: "Email address (required)" or include "(required)" in visible label
 - `aria-required="true"` for custom validation
- **Accessible error messages:**
 - Error messages must be programmatically associated with fields
 - Use `aria-describedby` to link error messages to inputs
 - Error messages should be clear and actionable
 - Bad: "Invalid input"
 - Good: "Email must include @ symbol, for example: [user@example.com](#)"

```
<label for="email">Email address (required)</label>
<input type="email" id="email" aria-describedby="email-error" required>
<span id="email-error" class="error">Email must include @ symbol</span>
```

- **Form validation patterns:**
 - Inline validation (as user types or leaves field)
 - Summary of errors at top of form
 - Focus management: move focus to first error
 - Success messages for completed submissions

Transitions: You've now learned to make core HTML elements accessible. The next section introduces ARIA attributes that extend accessibility beyond what HTML alone can provide.

Key Principles:

- Labels must be visible and persistent, not hidden or placeholder-based
- Every form field needs a label—no exceptions
- Error messages must be programmatically associated with the field, not just visually nearby
- Good error messages explain what's wrong AND how to fix it

Examples:

- **Bad:** Input with placeholder="Enter your email" but no label
- **Good:** Visible label "Email address" with separate placeholder showing format
- **Bad:** Error message "Error" in red text near the field
- **Good:** Error message "Email must include @ symbol" connected via `aria-describedby`

Hands-On Exercise:

Practice Challenge: Create an accessible contact form

Scenario: Build a simple contact form with proper labels, required field indicators, and error message support.

Tasks:

Create an HTML form with the following fields, ensuring each is fully accessible:

1. Name field (required)

- Text input
- Visible label
- Required indicator
- Error message span (prepared for validation)

2. Email field (required)

- Email input
- Visible label
- Required indicator
- Error message span with id
- Connected to input via `aria-describedby`

3. Message field (required)

- Textarea (minimum 10 characters)
- Visible label
- Required indicator
- Character count or length requirement indication

4. Preferred contact method (optional)

- Radio button group
- Options: Email, Phone
- Grouped with `<fieldset>` and `<legend>`
- Each radio button has its own label

5. **Newsletter checkbox** (optional)

- Checkbox input
- Clear label text explaining what user is agreeing to

6. **Submit button**

- Descriptive text (not just "Submit")

Requirements:

- All inputs have associated labels using `for` attribute
- Required fields marked in label text AND have `required` attribute
- Error message elements included with unique IDs
- Error messages connected to inputs via `aria-describedby`
- Radio group uses `<fieldset>` and `<legend>`
- No placeholder-only fields

Success Criteria:

- Can navigate entire form using only Tab key
- Clicking a label focuses its associated input
- Screen reader announces label, field type, required status, and error message
- Visual indicators for required fields are supplemented by text
- Form can be submitted (action="#" is acceptable for this exercise)

Expected Outcomes: Students will create a complete, accessible form demonstrating proper label association, required field indication, and error message structure.

Section 03: ARIA Attributes and Roles

03.0 Introduction to ARIA 📖 (~500 words)

Learning Objectives:

- Understand what ARIA is and when it should be used
- Recognize the three categories of ARIA attributes
- Apply the "First Rule of ARIA": Don't use ARIA if semantic HTML suffices

Content Outline:

- **What is ARIA?**
 - Accessible Rich Internet Applications
 - W3C specification for making dynamic web content accessible
 - Extends HTML semantics for complex interactions
 - Does not change functionality—only changes how assistive technologies interpret elements
- **The First Rule of ARIA:**
 - "If you can use a native HTML element or attribute, then do so"
 - Semantic HTML is always preferable
 - Example: Use `<button>` instead of `<div role="button">`
 - ARIA is for situations where HTML has no semantic solution
- **When ARIA is necessary:**
 - Custom widgets not available in HTML (tree views, tabs, carousels)
 - Dynamic content updates (live regions)
 - Complex application interfaces
 - Providing additional context beyond what HTML conveys
- **Three categories of ARIA attributes:**

- **Roles:** Define what an element is (`role="navigation"` , `role="button"`)
- **States:** Dynamic values that change (`aria-expanded="true"`)
- **Properties:** Relationships and attributes (`aria-labelledby="id"` , `aria-describedby="id"`)
- **ARIA does not:**
 - Make elements focusable (use `tabindex` for that)
 - Add keyboard behavior (you must code that with JavaScript)
 - Change visual appearance (use CSS for that)
 - Work automatically (requires proper implementation)

Transitions: ARIA is a powerful tool for extending accessibility beyond HTML's capabilities. In the next lesson, you'll learn the most common ARIA roles and how to apply them.

Key Principles:

- ARIA supplements HTML, it doesn't replace it
- Use semantic HTML first; add ARIA only when HTML is insufficient
- ARIA only changes what assistive technologies perceive, not how elements actually behave
- Incorrect ARIA is worse than no ARIA—it can mislead users

Examples:

- **Good use:** `<div role="alert">` for a dynamic notification not achievable with HTML
- **Bad use:** `<div role="button">` when `<button>` element is available
- **Good use:** `aria-live="polite"` to announce dynamic content updates
- **Bad use:** Adding `role="button"` to a link—use the correct element instead

03.1 Common ARIA Roles 📖 (~550 words)

Learning Objectives:

- Implement landmark roles to define page regions
- Use widget roles for interactive components
- Understand when roles duplicate semantic HTML elements

Content Outline:

- **Landmark roles** (define major page regions):
 - `role="banner"` - Main header (usually `<header>` suffices)
 - `role="navigation"` - Navigation section (usually `<nav>` suffices)
 - `role="main"` - Main content (usually `<main>` suffices)
 - `role="complementary"` - Supporting content (usually `<aside>` suffices)
 - `role="contentinfo"` - Footer info (usually `<footer>` suffices)
 - `role="search"` - Search functionality
 - Note: Modern semantic HTML elements have implicit ARIA roles
- **Widget roles** (interactive components):
 - `role="button"` - Clickable button (use `<button>` instead when possible)
 - `role="tab"` and `role="tabpanel"` - Tab interface
 - `role="dialog"` - Modal dialog
 - `role="alert"` - Important message needing immediate attention
 - `role="menu"` and `role="menuitem"` - Application menu (not website navigation)
- **Document structure roles:**
 - `role="heading"` with `aria-level="2"` (use `<h2>` instead)
 - `role="list"` and `role="listitem"` (use `<ul>` / `<ol>` instead)
 - `role="article"` (use `<article>` instead)
- **When to add explicit roles:**
 - When using generic elements (`<div>` , `<span>`) for semantic purposes

- When semantic HTML doesn't exist for the pattern (tabs, carousels)
- When overriding default semantics (rare and should be avoided)

Transitions: Roles define what elements are. Next, you'll learn about ARIA properties that provide labels and descriptions.

Key Principles:

- Semantic HTML5 elements have implicit ARIA roles—adding explicit roles is redundant
- Landmark roles help screen reader users navigate page structure quickly
- Widget roles require complete keyboard interaction implementation
- Don't override native element semantics unless absolutely necessary

Examples:

- Redundant: `<nav role="navigation">` (nav already has implicit role)
- Necessary: `<div role="navigation">` (div has no semantic meaning, role is needed)
- Necessary: `<div role="tab" tabindex="0">` (no HTML tab element exists)
- Bad: `<a role="button">` (changes semantics incorrectly)

Hands-On Exercise:

Practice Challenge: Add appropriate ARIA roles to a component

Scenario: You're building a simple tab interface using `<div>` elements. Since HTML has no native tab element, you'll use ARIA roles to make it accessible.

Starting HTML:

```
<div class="tab-container">
  <div class="tab-list">
    <div class="tab active">Profile</div>
    <div class="tab">Settings</div>
    <div class="tab">Messages</div>
  </div>

  <div class="tab-content active">
    <h2>Profile</h2>
    <p>Your profile information...</p>
  </div>

  <div class="tab-content">
    <h2>Settings</h2>
    <p>Adjust your preferences...</p>
  </div>

  <div class="tab-content">
    <h2>Messages</h2>
    <p>Your messages...</p>
  </div>
</div>
```

Tasks:

1. Add `role="tablist"` to the container holding the tabs
2. Add `role="tab"` to each clickable tab element
3. Add `role="tabpanel"` to each content area
4. Add `tabindex="0"` to make tabs keyboard focusable
5. Add `aria-selected="true"` to the active tab
6. Add `aria-selected="false"` to inactive tabs

7. Connect each tab to its panel using `aria-controls` (pointing to panel id)
8. Connect each panel to its tab using `aria-labelledby` (pointing to tab id)

Success Criteria:

- All ARIA roles correctly applied
- Tabs are keyboard focusable
- Active tab marked with `aria-selected="true"`
- Proper associations between tabs and panels
- Unique IDs for all tabs and panels to enable `aria-controls/labelledby`

Expected Outcomes: Students will understand how ARIA roles define component structure and how attributes create relationships between elements.

03.2 aria-label and aria-labelledby (~550 words)

Learning Objectives:

- Use `aria-label` to provide accessible names for elements
- Use `aria-labelledby` to reference existing text as a label
- Understand when each attribute is appropriate
- Recognize the difference between labels and descriptions

Content Outline:

- **aria-label fundamentals:**

- Provides an accessible name for an element
- Overrides visible text content
- Used when no visible label exists or visible text is insufficient
- Example: Icon buttons without visible text

```
<button aria-label="Close dialog">
  <span class="icon-x"></span>
</button>
```

- **aria-labelledby fundamentals:**

- References one or more elements by ID to create a label
- Uses existing visible text, avoiding redundancy
- Can combine multiple elements into one label

```
<h2 id="dialog-title">Confirm deletion</h2>
<div role="dialog" aria-labelledby="dialog-title">
  <p>Are you sure you want to delete this file?</p>
</div>
```

- **Choosing between aria-label and aria-labelledby:**

- Use `aria-labelledby` when visible text already exists
- Use `aria-label` when no visible label is appropriate
- `aria-labelledby` is preferable because it keeps visible and accessible text in sync

- **aria-describedby** (for additional context, not labels):

- Provides supplementary description
- Announced after the label and role
- Used for error messages, hints, help text

```
<input type="email" id="email"
  aria-label="Email address"
```

```
aria-describedby="email-hint">
<span id="email-hint">We'll never share your email</span>
```

- **Common use cases:**

- Icon-only buttons: `aria-label`
- Modal dialogs: `aria-labelledby` (pointing to modal title)
- Form fields with error messages: `aria-describedby`
- Complex widgets: combination of labeling strategies

Transitions: You've learned how to label elements with ARIA. Next, you'll learn when ARIA should and should not be used.

Key Principles:

- Visible text is always preferable—use `aria-label` only when necessary
- `aria-labelledby` keeps visual and accessible experiences in sync
- Every interactive element needs an accessible name (from label, `aria-label`, or `aria-labelledby`)
- Labels identify what something is; descriptions provide additional context

Examples:

- **Good `aria-label` use:** `<button aria-label="Search">🔍</button>` (icon-only button)
- **Bad `aria-label` use:** `<button aria-label="Click me">Submit</button>` (contradicts visible text)
- **Good `aria-labelledby`:** `<dialog aria-labelledby="title-id">` (references visible heading)
- **Good `aria-describedby`:** `<input aria-describedby="password-requirements">` (links to help text)

Hands-On Exercise:

Practice Challenge: Properly label elements using ARIA

Scenario: You're building a user profile card with several interactive elements. Some need `aria-label`, others need `aria-labelledby`, and some need `aria-describedby`.

Tasks:

Add appropriate ARIA labeling to this HTML:

```
<article class="profile-card">
  <h2 id="user-name">Sarah Chen</h2>
  <p id="user-bio">Senior Software Engineer</p>

  <!-- Social media links (icon-only buttons) -->
  <div class="social-links">
    <a href="https://twitter.com/sarahchen">
      
    </a>
    <a href="https://linkedin.com/in/sarahchen">
      
    </a>
    <a href="https://github.com/sarahchen">
      
    </a>
  </div>

  <!-- Edit profile button -->
  <button class="icon-button">
    <span class="pencil-icon"></span>
  </button>

  <!-- Delete account dialog -->
  <div role="dialog" class="hidden">
```

```

<h3 id="delete-title">Delete Account</h3>
<p>This action cannot be undone.</p>

<label for="confirm-text">
  Type "DELETE" to confirm
</label>
<input type="text" id="confirm-text">
<span id="confirm-help">This helps prevent accidental deletion</span>

<button>Cancel</button>
<button class="danger">Delete My Account</button>
</div>
</article>

```

Your tasks:

1. Add `aria-label` to each social media link (describing destination)
2. Add `aria-label` to the edit button (no visible text)
3. Add `aria-labelledby` to the dialog (pointing to its title)
4. Add `aria-describedby` to the confirmation input (pointing to help text)
5. Ensure all interactive elements have accessible names

Success Criteria:

- Each social link has descriptive `aria-label`
- Icon button has `aria-label` explaining function
- Dialog uses `aria-labelledby` referencing visible title
- Input field connected to help text via `aria-describedby`
- No generic "link" or "button" announcements remain

Expected Outcomes: Students will practice choosing the appropriate ARIA labeling technique for different scenarios and understand how labels and descriptions work together.

03.3 When to Use (and Not Use) ARIA 📖 (~600 words)

Learning Objectives:

- Apply the five rules of ARIA to make informed decisions
- Recognize situations where ARIA makes things worse
- Debug common ARIA mistakes

Content Outline:

- **The Five Rules of ARIA** (from W3C):
 1. **Don't use ARIA if HTML suffices:** `<button>` beats `<div role="button">`
 2. **Don't change native semantics:** Never `<h2 role="tab">` (use `div` with `role` instead)
 3. **All interactive ARIA controls must be keyboard accessible:** Adding `role` doesn't add behavior
 4. **Don't use `role="presentation"` or `aria-hidden="true"` on focusable elements:** Creates confusion
 5. **All interactive elements must have an accessible name:** From `label`, `aria-label`, or `aria-labelledby`
- **Situations where ARIA is appropriate:**
 - Custom widgets with no HTML equivalent (tabs, accordions, tree views)
 - Live regions announcing dynamic updates
 - Complex application interfaces (dashboards, email clients)
 - Adding context that visual users get but screen readers miss
- **Situations where ARIA is inappropriate:**
 - When semantic HTML exists: use `<nav>`, not `<div role="navigation">`
 - Trying to "fix" bad HTML: rewrite with proper elements instead

- Adding roles without keyboard support: role without behavior is misleading
- Overriding element semantics unnecessarily
- **Common ARIA mistakes:**
 - **Adding redundant roles:** `<button role="button">` (button already has this role)
 - **Role without behavior:** `<div role="button">` without keyboard handlers
 - **Conflicting semantics:** `<a role="button">` (confuses purpose)
 - **Hidden but focusable:** `<button aria-hidden="true">` (keyboard focus but screen reader ignores)
 - **No accessible name:** `<button role="tab"></button>` (what is this tab called?)
- **Testing ARIA implementation:**
 - Use screen reader to verify announced information
 - Ensure keyboard interaction matches ARIA pattern
 - Check that dynamic states update correctly
 - Validate with automated tools (but manual testing essential)

Transitions: You now understand when ARIA enhances accessibility and when it creates problems. The next section explores common accessibility issues and how to avoid them.

Key Principles:

- ARIA is a last resort, not a first choice
- Incorrect ARIA actively harms accessibility—it's worse than no ARIA
- Every ARIA role creates an expectation of behavior that must be fulfilled
- The best ARIA is the ARIA you don't need because you used semantic HTML

Examples:

- **Bad:** `<div role="button" aria-label="Submit">Submit</div>` → Should be `<button>Submit</button>`
 - **Good:** `<div role="tab" tabindex="0" aria-selected="true">Settings</div>` with full keyboard support
 - **Bad:** `<h2 role="tab">Tab Title</h2>` → Never change heading semantics
 - **Bad:** `<button aria-hidden="true">Click me</button>` → Focusable but announced as nothing
 - **Good:** `<div role="alert" aria-live="assertive">Form submitted successfully</div>` (no HTML alternative)
-

Section 04: Common Accessibility Problems

04.0 Color Contrast and Visual Issues (~450 words)

Learning Objectives:

- Understand WCAG color contrast requirements
- Identify common visual accessibility barriers
- Use tools to check and fix contrast issues

Content Outline:

- **WCAG contrast requirements:**
 - **Level AA (standard):** 4.5:1 for normal text, 3:1 for large text (18pt+ or 14pt+ bold)
 - **Level AAA (enhanced):** 7:1 for normal text, 4.5:1 for large text
 - Applies to text against backgrounds, including text on images
 - Interactive elements (buttons, links) also require 3:1 against adjacent colors
- **Why contrast matters:**
 - Low vision users
 - Color blindness (affects ~8% of men, ~0.5% of women)
 - Users in bright sunlight or poor lighting
 - Aging eyes (affects everyone eventually)
- **Common contrast problems:**
 - Light gray text on white backgrounds (#999 on #FFF = 2.8:1, fails)

- Placeholder text (often too light)
- Disabled button states (often exempted but should still be somewhat visible)
- Text over background images without sufficient overlay
- Link text that relies only on color (must also use underline or other visual distinction)
- **Other visual accessibility issues:**
 - **Small text sizes:** Minimum 16px for body text recommended
 - **Line height:** Minimum 1.5 for body text (WCAG SC 1.4.12)
 - **Line length:** 80 characters or less for optimal readability
 - **Color alone:** Never use color as the only way to convey information
 - **Flashing content:** Nothing flashes more than 3 times per second (seizure risk)
- **Tools for checking contrast:**
 - Browser DevTools color picker (shows contrast ratio)
 - WebAIM Contrast Checker
 - Colors contrast checker
 - Browser extensions (e.g., WAVE, Axe DevTools)

Transitions: Visual accessibility extends beyond screen readers to anyone viewing your content. Next, you'll practice identifying and fixing keyboard navigation problems.

Key Principles:

- Sufficient contrast benefits everyone, not just users with disabilities
- Color should enhance meaning, never be the sole conveyor of meaning
- Automated tools can check contrast ratios precisely—use them
- Design decisions about color affect accessibility from the very beginning

Examples:

- **Fails:** Light gray on white (#999 on #FFF = 2.8:1)
- **Passes AA:** Dark gray on white (#595959 on #FFF = 7:1)
- **Fails:** Red text to indicate errors without any other indicator (icon, text label)
- **Passes:** Red text + "Error:" prefix + warning icon to indicate errors

04.1 Keyboard Navigation Problems (~475 words)

Learning Objectives:

- Identify common keyboard navigation barriers
- Test website keyboard accessibility
- Understand tab order and focus management

Content Outline:

- **Why keyboard accessibility matters:**
 - Screen reader users navigate by keyboard
 - Users with motor disabilities may use keyboard, switch devices, or voice control
 - Power users prefer keyboard shortcuts
 - Touch interfaces (mobile) have their own keyboard
- **Keyboard navigation basics:**
 - **Tab:** Move forward through interactive elements
 - **Shift+Tab:** Move backward
 - **Enter/Space:** Activate buttons, links, form controls
 - **Arrow keys:** Navigate within components (radio groups, menus, sliders)
 - **Escape:** Close dialogs, menus, or cancel operations
- **Common keyboard navigation problems:**
 - **Non-focusable interactive elements:** `<div onclick="...">` without `tabindex`
 - **Keyboard traps:** Focus gets stuck in a modal or element, can't Tab out

- **Wrong tab order:** Visual layout doesn't match DOM order
- **Skip links missing:** No way to bypass repetitive navigation
- **Custom widgets without keyboard support:** Carousels, dropdowns that only work with mouse
- **Form controls requiring mouse:** Drag-and-drop without keyboard alternative
- **Testing keyboard accessibility:**
 - Put away your mouse
 - Try to complete every task using only keyboard
 - Can you reach every interactive element?
 - Can you see where focus is at all times?
 - Can you activate buttons, open menus, submit forms?
 - Can you escape from modal dialogs and overlays?
- **The tab order:**
 - Natural tab order follows DOM order, not visual layout
 - CSS positioning can create confusing visual vs. DOM order mismatches
 - Fix: Reorder in HTML, not with `tabindex` values > 0
 - `tabindex="0"` : Adds element to natural tab order
 - `tabindex="-1"` : Programmatically focusable but not in tab order
 - `tabindex="1+"` : Avoid—creates unpredictable order

Transitions: Keyboard access is essential, but only useful if users can see where they are. Next, you'll learn about focus indicators.

Key Principles:

- If you can't use your site with only a keyboard, it's not accessible
- Tab order should match the visual reading order
- Custom interactive elements require custom keyboard handlers
- Keyboard traps are serious accessibility violations

Examples:

- **Bad:** `<div onclick="openMenu()">Menu</div>` (not keyboard accessible)
- **Good:** `<button onclick="openMenu()">Menu</button>` (keyboard accessible by default)
- **Bad:** Modal dialog without Escape key handler (keyboard trap)
- **Good:** Modal that traps focus within dialog and closes on Escape

Hands-On Exercise:

Practice Challenge: Identify and fix keyboard navigation issues

Scenario: You're testing a website and discover several keyboard navigation problems. Document the issues and propose fixes.

Test these interactions:

1. Custom dropdown menu (HTML provided below)

```
<div class="dropdown">
  <span class="dropdown-trigger" onclick="toggleMenu()">Options</span>
  <div class="dropdown-menu hidden">
    <div class="menu-item" onclick="selectOption(1)">Option 1</div>
    <div class="menu-item" onclick="selectOption(2)">Option 2</div>
    <div class="menu-item" onclick="selectOption(3)">Option 3</div>
  </div>
</div>
```

2. Image carousel

- Previous/Next buttons work only on mouse click
- No keyboard way to advance slides

3. Modal dialog

- Opens when clicking a button
- Focus stays on button behind modal
- No way to close with keyboard

Tasks: For each problem:

1. Describe the keyboard accessibility issue
2. Explain which users are affected
3. Propose specific HTML/ARIA fixes
4. Identify what keyboard behaviors should work

Success Criteria:

- Correctly identified that `<span>` elements aren't keyboard focusable
- Recognized focus management issues in modal
- Proposed adding `tabindex`, `role`, and keyboard event handlers
- Understood that mouse-only controls exclude keyboard users

Expected Outcomes: Students will develop the ability to spot keyboard accessibility problems and understand the requirements for keyboard-accessible custom components.

04.2 Missing or Poor Focus Indicators (~475 words)

Learning Objectives:

- Understand why visible focus indicators are essential
- Identify insufficient focus indicators
- Implement custom focus styles that meet accessibility requirements

Content Outline:

- **What is a focus indicator?**
 - Visual indication of which element currently has keyboard focus
 - Browser default: usually a blue outline or border
 - Announces to sighted keyboard users: "You are here"
- **Why focus indicators matter:**
 - Sighted keyboard users (motor disabilities, power users) need to see current position
 - Without focus indicators, keyboard navigation is unusable
 - Equivalent to a mouse cursor disappearing
- **Common focus indicator problems:**
 - **Removed entirely:** `*:focus { outline: none; }` with no replacement
 - **Insufficient contrast:** Focus color too similar to background
 - **Too subtle:** 1px outline that's barely visible
 - **Lost in design:** Focus indicator blends into surrounding design
 - **Inconsistent:** Different styles across different elements
- **WCAG requirements for focus indicators** (SC 2.4.7, 2.4.11):
 - Must be visible (obvious difference from non-focused state)
 - Minimum 2px thick (or equivalent area)
 - Contrast ratio of at least 3:1 against adjacent colors
 - Consistent across the site
- **Designing custom focus indicators:**
 - Don't remove default outline without replacement
 - Make focus state at least as prominent as hover state
 - Use consistent pattern across all interactive elements
 - Consider both light and dark modes

- Test with actual keyboard navigation

```
/* Good: Custom but visible focus */
button:focus {
  outline: 3px solid #0066CC;
  outline-offset: 2px;
}

/* Bad: Removed without replacement */
button:focus {
  outline: none;
}
```

- **Focus-visible pseudo-class:**

- `:focus-visible` shows focus only for keyboard navigation
- Hides focus ring for mouse clicks but shows it for keyboard
- Modern CSS solution to "ugly outline" complaints

```
button:focus-visible {
  outline: 3px solid #0066CC;
  outline-offset: 2px;
}
```

Transitions: Focus indicators are just one common problem. The next lesson will explore a broader range of accessibility anti-patterns to avoid.

Key Principles:

- Never remove focus indicators without providing an equivalent replacement
- Focus indicators must be at least as visible as hover states
- Custom focus styles must meet WCAG contrast and size requirements
- `:focus-visible` provides a modern solution to focus indicator visibility

Examples:

- **Bad:** `outline: none;` with no alternative styling
- **Good:** `outline: 2px solid blue; outline-offset: 2px;`
- **Bad:** Light gray focus ring on white background (insufficient contrast)
- **Good:** Dark blue focus ring with 3:1 contrast ratio

Hands-On Exercise:

Practice Challenge: Design accessible focus indicators

Scenario: A designer removed all default focus outlines because they "looked ugly" and didn't replace them. You need to add custom focus indicators that meet WCAG requirements.

Starting CSS:

```
* {
  outline: none; /* Designer removed these! */
}

.btn-primary {
  background: #007BFF;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 4px;
```

```

}

.btn-primary:hover {
  background: #0056b3;
}

.link-text {
  color: #007BFF;
  text-decoration: none;
}

.link-text:hover {
  text-decoration: underline;
}

input[type="text"] {
  border: 1px solid #CCC;
  padding: 8px;
  border-radius: 4px;
}

```

Tasks:

1. Remove the global `outline: none;` rule
2. Add custom `:focus` styles for `.btn-primary` that:
 - Have at least 3:1 contrast against the button background
 - Are at least 2px thick
 - Have visual offset from button edge
3. Add `:focus` styles for `.link-text` that are clearly visible
4. Add `:focus` styles for `input[type="text"]` that differ from default state
5. Bonus: Implement `:focus-visible` for mouse vs. keyboard differentiation

Requirements:

- All focus indicators must be clearly visible
- Focus states must be at least as prominent as hover states
- Color contrast must meet 3:1 minimum
- Use `outline-offset` for spacing when appropriate

Success Criteria:

- Global outline removal is undone
- Each interactive element has visible focus indicator
- Focus indicators meet WCAG size and contrast requirements
- Styles are consistent across similar elements
- Testing with Tab key shows clear focus position

Expected Outcomes: Students will practice creating custom focus indicators that balance aesthetic design with accessibility requirements.

04.3 Anti-Pattern: Common Accessibility Mistakes 📖 (~500 words)

Learning Objectives:

- Recognize frequent accessibility anti-patterns in real websites
- Understand why these patterns create barriers
- Know the correct alternative for each anti-pattern

Content Outline:

- **Anti-Pattern 1: Placeholder as label**

- **The mistake:** Using placeholder text instead of a proper label

```
<!-- Wrong -->
<input type="email" placeholder="Enter your email">

<!-- Right -->
<label for="email">Email address</label>
<input type="email" id="email" placeholder="name@example.com">
```

- **Why it's wrong:** Placeholders disappear when typing, not announced by all screen readers
- **Impact:** Form fields become unidentifiable mid-completion

- **Anti-Pattern 2: Click here links**

- **The mistake:** Generic link text that makes no sense out of context

```
<!-- Wrong -->
To learn more, <a href="/about">click here</a>.

<!-- Right -->
<a href="/about">Learn more about our accessibility services</a>.
```

- **Why it's wrong:** Screen readers can list all links—"click here" repeated 20 times is useless
- **Impact:** Impossible to scan links for specific content

- **Anti-Pattern 3: Div/span as button**

- **The mistake:** Using non-semantic elements for interactive components

```
<!-- Wrong -->
<div onclick="submitForm()">Submit</div>

<!-- Right -->
<button onclick="submitForm()">Submit</button>
```

- **Why it's wrong:** Not keyboard accessible, no built-in ARIA, requires extensive fixes
- **Impact:** Keyboard users cannot interact, screen readers don't announce correctly

- **Anti-Pattern 4: Auto-playing media**

- **The mistake:** Videos or audio that play automatically with sound
- **Why it's wrong:** Screen reader audio competes with media, violates WCAG SC 1.4.2
- **Impact:** Screen reader users can't hear their assistive technology
- **Correct approach:** Autoplay only if muted, provide clear pause/stop control

- **Anti-Pattern 5: Color-only information**

- **The mistake:** Relying solely on color to convey meaning

```
<!-- Wrong -->
<span style="color: red;">Error</span>

<!-- Right -->
<span style="color: red;">⚠️ Error: Invalid email format</span>
```

- **Why it's wrong:** Color blind users cannot perceive the distinction
- **Impact:** Critical information (errors, status) becomes invisible

- **Anti-Pattern 6: Unlabeled icon buttons**

- **The mistake:** Icon-only buttons without accessible names

```
<!-- Wrong -->
<button><i class="icon-search"></i></button>

<!-- Right -->
<button aria-label="Search"><i class="icon-search"></i></button>
```

- **Why it's wrong:** Screen readers announce "button" with no indication of function
- **Impact:** Users don't know what the button does

- **Anti-Pattern 7: Opening new windows without warning**

- **The mistake:** Links that open new tabs/windows unexpectedly
- **Why it's wrong:** Disorients users, breaks back button, violates user expectation
- **Correct approach:** Indicate new window in link text or with icon + aria-label

Transitions: You've now learned what not to do. The next section will help you test for these problems and verify your fixes.

Key Principles:

- Most accessibility anti-patterns stem from using the wrong HTML element or no semantic HTML
- Anti-patterns often prioritize visual design over functional accessibility
- Each anti-pattern has a straightforward correction using semantic HTML or proper ARIA
- These mistakes are common and easily avoided with accessibility awareness

Examples: Each anti-pattern shown with wrong/right examples above demonstrates real accessibility barriers that can be fixed with simple changes.

Section 05: Testing and Assessment

05.0 Testing with Browser Tools and Screen Readers (~750 words)

Learning Objectives:

- Perform basic screen reader testing on your own websites
- Use browser accessibility inspection tools effectively
- Create a practical accessibility testing workflow
- Interpret and prioritize accessibility issues found during testing

Content Outline:

- **Setting up for accessibility testing:**
 - Browser DevTools: Chrome, Firefox, Edge all have accessibility inspectors
 - Screen readers: NVDA (Windows, free), VoiceOver (macOS, built-in), Narrator (Windows, built-in)
 - Browser extensions: axe DevTools, WAVE, Accessibility Insights
 - Testing on actual devices when possible (mobile screen readers)
- **Browser DevTools accessibility features:**
 - **Chrome DevTools:**
 - Accessibility pane: Shows ARIA attributes, computed accessibility tree
 - Lighthouse: Automated accessibility audit with specific recommendations

- Color contrast checker in color picker
 - Vision deficiency emulation
- **Firefox Accessibility Inspector:**
 - Shows accessibility tree
 - Highlights tabbing order
 - Checks for text label issues
 - Simulates various vision conditions
- **Basic screen reader testing workflow:**
 - **Before testing:** Understand normal screen reader behavior
 - **During testing:**
 1. Turn on screen reader
 2. Navigate to your page
 3. Use Tab key to navigate interactive elements
 4. Use H key to navigate headings
 5. Use landmarks to navigate page regions
 6. Fill out and submit forms
 7. Interact with custom components
 - **What to listen for:**
 - Are all elements announced with appropriate roles?
 - Do all images have meaningful alt text?
 - Are form fields properly labeled?
 - Do error messages get announced?
 - Can you complete all tasks without seeing the screen?
- **NVDA basic commands** (Windows):
 - NVDA + N: Open NVDA menu
 - Insert: NVDA modifier key
 - Insert + Down arrow: Read all from current position
 - H: Next heading
 - K: Next link
 - F: Next form field
 - B: Next button
 - Tab: Next interactive element
- **VoiceOver basic commands** (macOS):
 - Cmd + F5: Turn VoiceOver on/off
 - VO = Control + Option (VoiceOver modifier)
 - VO + A: Start reading
 - VO + Right/Left Arrow: Move to next/previous item
 - VO + H: Next heading
 - VO + Command + H: Open headings menu
 - Tab: Next interactive element
- **Creating an accessibility testing checklist:**
 - ☐ Keyboard navigation: Can reach and operate all interactive elements
 - ☐ Focus indicators: Visible on all focusable elements
 - ☐ Screen reader: All content announced appropriately
 - ☐ Headings: Logical hierarchy, no skipped levels
 - ☐ Alt text: All images have meaningful alternatives
 - ☐ Forms: All fields labeled, errors announced
 - ☐ Color contrast: Meets WCAG AA (4.5:1 minimum)

- ☐ No keyboard traps: Can escape from all components
- ☐ Zoom: Page works at 200% zoom
- ☐ Responsive: Works on mobile devices

- **Interpreting automated test results:**

- **Errors:** Must fix (clear WCAG violations)
- **Warnings:** Need manual review (may or may not be problems)
- **Features to review:** Manual verification needed (e.g., is alt text meaningful?)
- **Passed:** No issues detected (but remember: tools catch only ~30-40%)

- **Prioritizing fixes:**

1. **Critical:** Prevents page use (keyboard traps, missing form labels, no alt on critical images)
2. **High:** Significant barriers (poor contrast, missing headings, unclear errors)
3. **Medium:** Creates difficulty (unclear link text, minor contrast issues)
4. **Low:** Best practice violations (redundant ARIA, non-semantic HTML)

Transitions: You've learned to test for accessibility issues. Now it's time to assess your understanding of all the concepts covered in this course.

Key Principles:

- Automated testing finds obvious problems; manual testing finds usability problems
- Screen reader testing doesn't require expertise—basic testing catches most issues
- Testing should happen throughout development, not just at the end
- Real users with disabilities provide the most valuable testing feedback

Examples:

- Running Lighthouse and getting 85/100 with recommendations for missing alt text and insufficient contrast
- Using Tab key and discovering a modal dialog you can't escape from
- Testing with NVDA and hearing "button" announced without any label
- Using WAVE and seeing form labels correctly associated with green checkmarks

Hands-On Exercise:

Practice Challenge: Complete accessibility testing workflow

Scenario: You've built a simple registration form. Test it for accessibility using multiple methods and document your findings.

HTML to test:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Create Account</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 500px; margin: 50px auto; }
    .form-group { margin-bottom: 20px; }
    label { display: block; margin-bottom: 5px; color: #666; }
    input { width: 100%; padding: 8px; border: 1px solid #ccc; }
    .error { color: #cc0000; font-size: 14px; }
    button { background: #007bff; color: white; padding: 12px 24px; border: none; }
    .required::after { content: " *"; color: red; }
  </style>
</head>
<body>
  <h1>Create Account</h1>
```

```

<form>
  <div class="form-group">
    <label for="username" class="required">Username</label>
    <input type="text" id="username" required>
  </div>

  <div class="form-group">
    <label for="email" class="required">Email</label>
    <input type="email" id="email" required>
    <span class="error">Please enter a valid email address</span>
  </div>

  <div class="form-group">
    <label for="password" class="required">Password</label>
    <input type="password" id="password" required>
  </div>

  <div class="form-group">
    <input type="checkbox" id="terms" required>
    <label for="terms">I agree to the terms and conditions</label>
  </div>

  <div class="form-group">
    <button type="submit">Create Account</button>
  </div>
</form>
</body>
</html>

```

Testing tasks:

1. Automated testing:

- Run Chrome Lighthouse accessibility audit
- Install and run WAVE or axe DevTools
- Document all errors and warnings found

2. Manual keyboard testing:

- Navigate form using only Tab key
- Can you complete and submit the form?
- Are focus indicators visible on all fields?
- Document any keyboard navigation issues

3. Screen reader testing (NVDA or VoiceOver):

- Navigate form with screen reader
- Are all labels announced?
- Is the error message associated with the email field?
- Are required fields indicated accessibly?

4. Visual inspection:

- Check color contrast of label text (#666 on white)
- Check visibility of required field indicators
- Check error message color contrast

Deliverables:

Create a test report including:

1. **Automated findings:** Issues found by Lighthouse/WAVE
2. **Keyboard navigation results:** Pass/fail for each element
3. **Screen reader observations:** What was announced correctly/incorrectly
4. **Accessibility issues found** (categorized as Critical/High/Medium/Low)
5. **Recommended fixes** for each issue

Success Criteria:

- Completed testing with at least 2 different methods
- Identified at least 5 accessibility issues
- Correctly prioritized issues by severity
- Proposed specific, actionable fixes
- Demonstrated understanding of WCAG principles

Expected Outcomes: Students will develop a practical accessibility testing workflow and learn to identify and prioritize real accessibility issues using multiple testing methods.

05.1 Accessibility Knowledge Check 🗨️ (~650 words)

Learning Objectives:

- Demonstrate understanding of core accessibility principles
- Apply WCAG guidelines to real-world scenarios
- Identify accessibility issues and propose solutions
- Connect accessibility concepts to practical implementation

Question Format: Scenario-based questions (7 questions)

Evaluation Criteria:

- 6-7 correct: Excellent understanding of accessibility fundamentals
- 4-5 correct: Good grasp of concepts with some areas to review
- 2-3 correct: Basic understanding; review ARIA and testing sections
- 0-1 correct: Needs to revisit course material before proceeding

Score Interpretation: This assessment evaluates your ability to apply accessibility principles to real situations. Focus areas needing review will be indicated based on which questions you miss.

Question 1: POUR Principles Application

You're reviewing a website where:

- Images display data visualizations without text alternatives
- The contact form can only be submitted by clicking a button (no Enter key support)
- Error messages appear in red text only (no icons or text labels)
- The site was built with divs and inline styles (no semantic HTML)

Which POUR principle is violated by EACH issue?

- A) Images: Perceivable | Form: Operable | Errors: Understandable | HTML: Robust B) Images: Robust | Form: Operable | Errors: Perceivable | HTML: Understandable
- C) Images: Perceivable | Form: Understandable | Errors: Understandable | HTML: Operable D) Images: Operable | Form: Perceivable | Errors: Robust | HTML: Understandable

Correct Answer: A

Explanation: Images without alt text aren't perceivable to screen reader users. Forms requiring mouse clicks aren't operable by keyboard users. Color-only errors aren't understandable to color-blind users. Non-semantic HTML isn't robust for assistive technologies.

Question 2: Semantic HTML vs. ARIA

A developer wants to create a navigation menu and considers these options:

```
<!-- Option 1 -->
<div role="navigation">
  <a href="/">Home</a>
  <a href="/about">About</a>
</div>

<!-- Option 2 -->
<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
</nav>

<!-- Option 3 -->
<nav role="navigation">
  <a href="/">Home</a>
  <a href="/about">About</a>
</nav>
```

Which is the best choice following accessibility best practices?

- A) Option 1 - Uses ARIA role for compatibility
- B) Option 2 - Uses semantic HTML without redundancy
- C) Option 3 - Combines both for maximum accessibility
- D) All options are equally accessible

Correct Answer: B

Explanation: The `<nav>` element has an implicit `role="navigation"`, making Option 2 correct per the First Rule of ARIA: "If semantic HTML exists, use it." Option 3 is redundant but not wrong. Option 1 requires ARIA where HTML suffices.

Question 3: Alt Text Best Practices

You're adding images to a blog post about accessible web design. Match each image to its appropriate alt text:

Images:

1. Company logo in the header (links to homepage)
2. Decorative geometric pattern separating sections
3. Screenshot showing a red error message: "Email is required"
4. Headshot of the author, Sarah Chen

Alt text options: A) `alt=""`

B) `alt="Sarah Chen"`

C) `alt="Screenshot of form validation error stating 'Email is required'"`

D) `alt="Company Name Home"`

Correct matching:

- A) Logo: D, Pattern: A, Screenshot: C, Headshot: B
- B) Logo: A, Pattern: B, Screenshot: C, Headshot: D

- C) Logo: D, Pattern: C, Screenshot: A, Headshot: B
D) Logo: B, Pattern: A, Screenshot: D, Headshot: C

Correct Answer: A

Explanation: Functional images (logo) describe the action/destination. Decorative images get empty alt. Informative images (screenshot) describe the information shown. Photos of people use their names.

Question 4: Form Accessibility

Which form implementation is most accessible?

```
<!-- Option A -->
<input type="email" placeholder="Enter your email address">

<!-- Option B -->
<input type="email" aria-label="Email address">

<!-- Option C -->
<label for="email">Email address</label>
<input type="email" id="email" placeholder="name@example.com">

<!-- Option D -->
<span>Email address</span>
<input type="email">
```

- A) Option A - Placeholder provides clear guidance
B) Option B - ARIA label works for all users
C) Option C - Visible label with placeholder for format example
D) Option D - Text label is visible to all users

Correct Answer: C

Explanation: Visible labels are always preferable. Placeholders disappear when typing. Option C provides a persistent visible label (accessible to all users) plus a placeholder for format guidance. Option B uses ARIA unnecessarily when HTML labels suffice.

Question 5: Keyboard Navigation

A developer created a custom dropdown menu:

```
<div class="dropdown">
  <span class="trigger" onclick="toggleMenu()">Menu</span>
  <div class="menu">
    <div onclick="selectItem(1)">Option 1</div>
    <div onclick="selectItem(2)">Option 2</div>
  </div>
</div>
```

What accessibility issues exist, and what fixes are needed?

- A) No issues - JavaScript handles everything
B) Needs `role="menu"` on wrapper and `role="menuitem"` on options
C) Needs `tabindex="0"` on trigger, `role="button"`, keyboard event handlers, and focus management
D) Needs only `tabindex` attributes to enable keyboard access

Correct Answer: C

Explanation: The trigger isn't keyboard focusable (needs tabindex), doesn't announce correctly (needs role), and doesn't respond to keyboard (needs Enter/Space handlers). Menu items also need focus management and keyboard support. Adding only ARIA (Option B) without behavior doesn't help.

Question 6: Color Contrast

You're choosing text colors for a website. Which combinations meet WCAG Level AA requirements (4.5:1 for normal text)?

Test each on a white background (#FFFFFF):

1. Dark gray text (#595959)
2. Light gray text (#999999)
3. Medium blue text (#0066CC)
4. Pale yellow text (#FFFFCC)

- A) Only 1 and 3 pass
- B) 1, 2, and 3 pass
- C) Only 1 passes
- D) All pass except 4

Correct Answer: A

Explanation: #595959 on white = 7:1 (passes), #999999 on white = 2.8:1 (fails), #0066CC on white = 7.5:1 (passes), #FFFFCC on white = 1.1:1 (fails). Only dark gray and medium blue provide sufficient contrast.

Question 7: ARIA Labeling

For an icon-only social media button, which is the best approach?

```
<!-- Option 1 -->
<button>
  
</button>

<!-- Option 2 -->
<button aria-label="Follow us on Twitter">
  
</button>

<!-- Option 3 -->
<button>
  
</button>

<!-- Option 4 -->
<button title="Follow us on Twitter">
  
</button>
```

- A) Option 1 - Alt text on image provides the label
- B) Option 2 - aria-label on button with empty alt on decorative icon
- C) Option 3 - Image speaks for itself
- D) Option 4 - Title attribute provides accessible name

Correct Answer: B

Explanation: `aria-label` on the button provides a clear, descriptive accessible name. The icon is decorative (the button is the functional element), so empty alt prevents redundant announcement. Option 1 works but is less descriptive. Option 4 relies on

Section 06: Building Accessible Websites

06.0 Your Accessibility Journey Continues (~450 words)

Content Outline:

What you've accomplished:

Congratulations! You've completed the fundamentals of web accessibility. You now understand:

- The POUR principles that guide all accessibility work
- How assistive technologies interpret web content
- Semantic HTML's role as the foundation of accessibility
- When and how to use ARIA attributes appropriately
- Common accessibility problems and how to avoid them
- Testing methods to verify your implementations

These skills form the foundation for creating inclusive web experiences that work for everyone.

The bigger picture:

Web accessibility isn't a checkbox to tick off—it's an ongoing practice that improves with experience. Every website you build, every component you create, is an opportunity to consider how diverse users will interact with your work. The habits you develop now will serve you throughout your career.

Remember: accessibility benefits everyone. The semantic HTML structure that helps screen reader users also improves SEO. The keyboard navigation you implement helps power users work more efficiently. The clear error messages that assist users with cognitive disabilities also reduce support requests. Accessibility is simply good design.

Next steps in your learning:

As you continue your web development journey, keep building your accessibility skills:

1. **Practice constantly:** Apply accessibility principles to every project
2. **Test with real users:** If possible, watch people with disabilities use websites
3. **Stay current:** WCAG guidelines evolve, new patterns emerge
4. **Explore advanced topics:**
 - Advanced ARIA patterns (comboboxes, tree views, live regions)
 - Accessible single-page applications
 - Mobile accessibility
 - Accessibility in JavaScript frameworks (React, Vue, Angular)
5. **Get involved:** Join accessibility communities, contribute to open source, share what you learn

Resources for continued learning:

- **W3C Web Accessibility Initiative (WAI):** Official WCAG documentation and tutorials
- **WebAIM:** Practical articles, guides, and the WAVE tool
- **A11y Project:** Community-driven accessibility resources
- **Deque University:** Comprehensive accessibility training
- **Screen reader documentation:** NVDA, JAWS, VoiceOver user guides
- **MDN Web Docs:** Accessibility section with practical examples

Your accessibility toolkit:

Bookmark these essential tools you've learned:

- Chrome Lighthouse (built into DevTools)

- WAVE browser extension
- axe DevTools
- Color contrast checkers
- NVDA or VoiceOver for testing

A final thought:

Accessibility is about people. Behind every technical requirement is a real person trying to access information, complete a task, or connect with others. By building accessible websites, you're not just following guidelines—you're removing barriers and creating opportunities for millions of people.

The web belongs to everyone. Thank you for committing to keep it that way.

Keep learning, keep testing, and keep building for everyone.